

Unidad N° 1: Ingeniería de Software en Contexto

1. Introducción a la Ingeniería del Software: ¿Qué es?

Software: set de programas y la documentación que los acompaña (IEEE). Incluye archivos de configuración, documentos de usuario, documentos de estructura del sistema y las herramientas usadas para construirlo.

- Tres tipos básicos: **software de sistema, utilitarios y software de aplicación**(el q trabajamos).
- Definición integradora (Pantaleo & Rinaudo, 2018): *el software es conocimiento representado en distintos niveles de abstracción* (desde los requerimientos, lo más abstracto, hasta el código, lo más detallado).

Ingeniería de Software:

Disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

Definiciones:

1. **IEEE (SWEBOK):** la **aplicación de un enfoque sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento del software; es decir, la aplicación de la ingeniería al software.
2. **Sommerville:** disciplina de la ingeniería que se ocupa de **todos los aspectos de la producción de software**, desde la concepción inicial hasta su operación y mantenimiento.
3. **David Parnas:** La definió de forma simple como la "**construcción de software multi-versión por parte de múltiples personas**" (*multi-person construction of multi-version software*).

Por qué importa: cada vez se necesitan sistemas confiables, rápidos y económicos; y a largo plazo es más barato usar métodos de IS, porque la **mayor parte del costo está en cambiar el software una vez en operación**. Primero hay que entender el problema; el diseño es una actividad crucial (da calidad y facilidad de mantenimiento).

Cuerpos de conocimiento:

- **SWEBOK** (Software Engineering Body of Knowledge): versión 3.0 de 2014, IEEE, **15 áreas de conocimiento**.
- **SEBoK** (Systems Engineering Body of Knowledge): versión 1.9.1 de 2018. Partes: Fundamentos de la Ing. de Sistemas; Ingeniería de Sistemas y Gestión (modelos de ciclo de vida, definición del concepto, definición/realización/despliegue del sistema, gestión); Aplicaciones (sistemas de producto, de servicio, de empresa, sistemas de sistemas, de salud); Facilitadores (negocios y empresas, equipos, individuos); Disciplinas relacionadas (con SW Eng, Project Mgmt, Industrial Eng, Speciality Eng).

5 Razones para no comparar el software con la manufactura tradicional

1. El software es **menos predecible**.
2. **No hay producción en masa**; casi ningún producto de software es idéntico a otro (los req. cambian).
3. **No todas las fallas son errores** de construcción.
4. El software **no se gasta** con el uso.
5. El software **no está gobernado por las leyes de la física**.

Esto demuestra que la creación de software es un esfuerzo puramente intelectual y creativo de un equipo humano, cuyo éxito radica en las personalidades e intelecto de sus miembros y no en la mera automatización de un proceso repetitivo. **Conclusión de la cátedra: saber programar NO es suficiente.**

Atributos del buen software: El buen software debe entregar la funcionalidad y el desempeño requeridos al usuario, además de ser **sustentable (mantenible), confiable y utilizable**.

2. Estado Actual y Antecedentes: La Crisis del Software

Un poco de historia del software

La historia muestra cómo la disciplina fue respondiendo a la crisis: primero con **métodos estructurados**, después con **modelos de madurez de procesos** (CMM/CMMI, procesos definidos) y finalmente con **Ágil y Lean** (procesos empíricos).

Año	Hito	Por qué importa
1968	Nace el término “Ingeniería de Software” en la conferencia de la NATO	Se reconoce la crisis del software y se propone aplicar ingeniería para enfrentarla
1975	The Mythical Man-Month , Frederick Brooks	Agregar gente a un proyecto atrasado lo atrasa más; hombres y meses no son intercambiables
1978	Tom DeMarco introduce el Análisis Estructurado	Primeros métodos formales para analizar y especificar sistemas. Predecesor de POO.
Años 80	Primeros grandes errores de software conocidos	El costo de las fallas se hace visible (ej. gusano de Morris, 1988)
1987	No Silver Bullet , Brooks	Distingue dificultades esenciales y accidentales : ninguna tecnología sola mejora 10x la productividad
1989	Managing the Software Process , Watts Humphrey	Base de la gestión y mejora de procesos; antecedente del CMM
1990	Internet y Orientación a Objetos	Cambian la escala y la forma de construir software
1991 / 1993	CMM 1.0 / CMM 1.1 (SEI)	Modelo de madurez de procesos: auge de los procesos definidos
2000	CMMI 1.0	Integra los modelos de madurez
2001	Manifiesto Ágil	Valores y principios ágiles; procesos empíricos frente a los definidos
2003	Lean Software Development (Poppendieck)	Adapta Lean (Toyota) al software: eliminar desperdicio

Contexto actual: las empresas más grandes del mundo son de software/tecnología (2026: NVIDIA, Alphabet, Apple, Microsoft, Amazon, TSMC, Broadcom, Saudi Aramco, Meta, Tesla). El software es central en la economía, pero sigue fallando.

Crisis del SW: término planteado en la conferencia de la OTAN en 1968 por Friedrich Bauer, quien recalcó la enorme dificultad de generar software libre de defectos, fácilmente comprensible y verificable.

“Algunos problemitas” con el desarrollo de software:

- La versión final no satisface las necesidades del cliente.
- No es fácil extenderlo ni adaptarlo; agregar funcionalidad en otra versión es casi imposible.
- Mala documentación y mala calidad.
- Más tiempo y costo que lo presupuestado.

Causas de la crisis:

- El hardware avanzó mucho más rápido que las técnicas de desarrollo; el software es una actividad humana, abstracta e intangible; crece la demanda de sistemas complejos; se subestima su complejidad; los productos deben cambiar constantemente; faltaba una disciplina que cubriera toda la producción. Hoy persiste por demandas crecientes (sistemas más grandes y rápidos) y bajas expectativas (empresas que no aplican IS y se conforman con que “funcione”).

Cuando nos va bien es por (Standish Group):

#	Factor de éxito	%
1	Involucramiento del usuario	15,9
2	Apoyo de la gerencia	13,0
3	Enunciado claro de los requerimientos	9,6
4	Planeamiento adecuado	8,2
5	Expectativas realistas	7,7
6	Hitos intermedios	7,7
7	Personas involucradas competentes	7,2

Cuando nos va mal es por:

#	Factor de fracaso	%
1	Requerimientos incompletos	13,1
2	Falta de involucramiento del usuario	12,4
3	Falta de recursos	10,6
4	Expectativas poco realistas	9,3
5	Falta de apoyo de la gerencia	8,7
6	Requerimientos cambiantes	8,1

Acá hay aspectos que son las premisas para los frameworks ágiles.

Truco para recordar: el top de ambos lados es el mismo al revés — **usuario involucrado + requerimientos claros + apoyo gerencial**. Casi ninguna causa es tecnológica.

3. Disciplinas que conforman la Ingeniería de Software

Se divide en tres áreas principales que interactúan para lograr la entrega de un producto:

- **Disciplinas Técnicas:** Orientadas a la construcción del software como producto final. Incluyen: Toma de requerimientos, análisis de requerimiento, diseño de sw, implementación de sw (codificación), **prueba de sw (testing)**, despliegue, documentación y capacitación al usuario.
- **Disciplinas de Gestión:** Orientadas a la **planificación, monitoreo y control del proyecto** del desarrollo de sw. Definen cómo se adaptará el proceso a las personas y al contexto. Se utilizan, por ejemplo, metodologías LEAN Agile de gestión de proyectos.
- **Disciplinas de Soporte:** son disciplinas transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software. Incluyen: **Gestión de Configuración de Software (SCM), Métricas y Aseguramiento de Calidad (QA)**. (paraguas)

4. Ejemplos de Grandes Proyectos de Software Fallidos y Exitosos

- **Proyectos Fallidos Históricos:**
 - *Mariner I (1962)*: Destruída por una fórmula matemática mal programada (\$18.5M).
 - *Hartford Coliseum (1978)*: Derrumbe del techo por cálculo erróneo en software CAD (\$70M).
 - *Ariane 5 (1996)*: Explosión por desbordamiento al convertir un dato de 64 a 16 bits (\$500M).
 - *Mars Climate Orbiter (1999)*: Pérdida de la sonda por desacople entre unidades inglesas y métricas (\$655M).
 - *Toyota ABS (2010)*: Retiro de 400.000 vehículos por retraso de software en el frenado (\$3.000M).
- **Proyectos Exitosos:**
 - *Apollo 11 (Margaret Hamilton)*: Software de navegación ultra-robusto con gestión de prioridades en tiempo real.

Los fracasos no suelen deberse a la tecnología sino a requerimientos, comunicación y gestión; los éxitos combinan usuario involucrado, validación temprana y entrega incremental.

5. No Silver Bullet y The Mythical Man-Month (Brooks)

No hay balas de plata (1987)

El ensayo plantea que el desarrollo de software es inherentemente complejo, y descarta la existencia de una "bala de plata" (solución única) capaz de aumentar la productividad o fiabilidad del software. La parte difícil de construir software es la **especificación, el diseño y la prueba de los constructos conceptuales**, no escribir código.

Para explicarlo, Brooks divide las dificultades en dos categorías:

A) Esenciales (Inherentes al software, imposibles de eliminar):

Tienen que ver con la naturaleza conceptual **abstracta** de lo que se construye. Son "Incurables"; es lo que queda hoy. Son cuatro propiedades:

1. **Complejidad:** Escalar software aumenta los elementos interactuando de forma no lineal. No hay piezas repetidas a nivel macro. No existen piezas repetidas a nivel macro como en la manufactura física. Esto provoca fallas de comunicación, estimación y control de costos.
2. **Conformidad:** Debe adaptarse a interfaces humanas arbitrarias y sistemas sin principios unificadores, dictados por el entorno.
3. **Mutabilidad:** Al ser "materia de pensamiento", el software es infinitamente maleable y está sometido a una presión de cambio continuo a lo largo del tiempo que ningún producto físico sufre.
4. **Invisibilidad:** El software carece de representación espacial o geométrica. No se puede dibujar en un plano o visualizar de forma directa, lo que dificulta enormemente que la mente humana lo conceptualice y modele sin perder de vista su integridad.

B) Dificultades Accidentales (No inherentes a la esencia del software):

Son las barreras asociadas a los aspectos de representación y producción del software (sintaxis del lenguaje, tiempos de compilación, limitaciones de hardware). Ya están casi resueltas; fáciles de automatizar:

Avances que solo atacaron lo accidental:

- **Lenguajes de programación de alto nivel:** Liberó al programador de lidiar con bits, registros y hardware.
- **Preservación de la inmediatez:** Eliminó el olvido de las minucias provocado por la compilación batch.
- **Entornos Integrados (IDES):** Facilitó la interconexión de programas y formatos.

Tampoco son balas de plata: POO, IA, sistemas expertos, programación automática y gráfica, verificación de programas, estaciones de trabajo potentes. Atacan la *representación*, no la creación del constructo mental.

Relectura moderna (IA):

El cuello de botella humano es **decidir el "qué"**. El prompt es el nuevo lenguaje de alto nivel: si el humano no entiende la complejidad esencial, el prompt será incompleto ("embudo de la especificación"). La IA nos dio velocidad para recorrer el laberinto, pero no el mapa: el diseño sigue siendo un acto humano.

¿Cómo atacar verdaderamente la esencia?

Dado que las dificultades accidentales están casi totalmente resueltas, Brooks sugiere enfocarse en estas tácticas conceptuales humanas:

- **Comprar antes que construir (Buy vs. Build):** Comprar frente a construir. Evitar construir por completo. La integración es más barata que generar código nuevo (incluso con IA).
- **Prototipado rápido (Rapid Prototyping):** Extraer requerimientos iterativamente. La IA brilla aquí solo como herramienta de descarte rápido para refinar la especificación.
- **Desarrollo Incremental ("Grow, not build" / Crecer, no construir):** Hacer correr primero un esqueleto mínimo que funcione e ir engrosando el sistema de forma orgánica, manteniendo siempre un producto funcional.
- **Grandes Diseñadores:** El talento humano es irremplazable. El salto de orden de magnitud sólo proviene de cultivar a las mejores mentes de diseño conceptual.

6. Proceso de software: procesos de desarrollo empíricos vs. definidos

Proceso (IEEE): la secuencia de pasos ejecutados para un propósito dado.

Proceso de Software: conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

- Conjunto estructurado de actividades que nos permiten en base a un conjunto de entradas producir alguna salida; varían según la organización y el tipo de sistema.



- **Debe ser explícitamente modelado si va a ser administrado.**
- Dentro de un proceso de sw vamos a encontrar: *procedimientos y métodos* · *personas con habilidades, entrenamiento y motivación* · *herramientas y equipos* (soporte tecnológico necesario para ejecutar el proceso). Las personas son lo principal: el software es una actividad **humano-intensiva**.
- Actividades fundamentales (Sommerville): especificación, desarrollo (diseño + programación), validación y evolución.
- No hay proceso ideal: sistemas críticos → proceso muy estructurado; sistemas de negocio con requerimientos cambiantes → proceso flexible.

	Proceso definido	Proceso empírico
Inspiración	Líneas de producción	La experiencia (de negocio, técnica), el contexto de las personas y el aprendizaje.
Supuesto	Se puede repetir el mismo proceso indefinidamente y obtener los mismos resultados (determinista)	Procesos complejos con <u>variables cambiantes</u> : al repetirlo se pueden obtener resultados distintos
Administración y control	Viene de la predictibilidad del proceso	Por inspecciones frecuentes y adaptaciones
Sirve para	Contextos estables, repetibles	Trabajo creativo y complejo (como el software)
Ciclo de vida	Cualquiera (secuencial, iterativo, recursivo). Ej.: PUD/RUP son definidos aunque iterativos	Iterativo-incremental (el secuencial no es viable)
Experiencia	Se asume extrapolable a otros proyectos	No es extrapolable : vale para ese equipo/proyecto
Punto de mejora	La mejora se logra optimizando el proceso : se analiza cada etapa intermedia y se estandariza para que sea más eficiente y predecible. Se apoya en el proceso , no en las personas.	La mejora se hace con ciclos cortos de retroalimentación : el equipo parte de una hipótesis, construye, inspecciona el resultado, lo contrasta con lo que suponía y adapta el proceso. Se apoya en las personas : la experiencia que gana el equipo en cada ciclo se hace explícita y compartida (conocimiento implícito → explícito) y se vuelve la base de la mejora.

Pilares del empirismo: transparencia (nos permite crecer como equipo y transformar el conocimiento implícito en explícito), **inspección** y **adaptación**. La transparencia permite inspeccionar; la inspección sin adaptación es inútil.

Patrón de conocimiento en procesos empíricos (ciclo): Asumir → Construir → Retroalimentar → Revisar → Adaptar , y vuelta a empezar. Es la forma de generar conocimiento cuando el resultado no se puede predecir. Se parte de una suposición, se construye algo chico para ponerla a prueba, se obtiene información real y se revisa gracias a la transparencia y la inspección. Con lo aprendido se ajusta el rumbo, que es la adaptación. Cada vuelta reduce la incertidumbre, porque el conocimiento no está al inicio sino que se construye con la experiencia; por eso estos procesos requieren ciclos de vida iterativos.

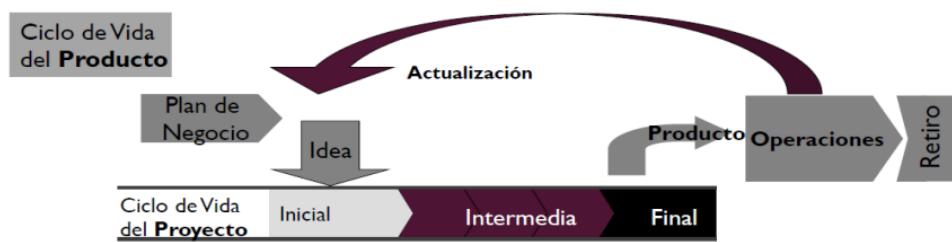
7. Ciclos de vida (modelos de proceso), procesos de desarrollo y su influencia en la administración

Ciclo de vida: la serie de pasos a través de los cuales el producto o el proyecto progresa. **Los productos tienen su ciclo de vida; los proyectos también.**

Ciclo de vida de un proyecto de software: una representación de un proceso; grafica una descripción del proceso desde una perspectiva particular. Los modelos especifican:

- **Las fases** del proceso (requerimientos, especificación, diseño...).
- **El orden** en que se llevan a cabo (y los criterios para pasar de una a otra).

Ciclo de vida del proyecto vs. del producto: el del producto es más largo (Idea → Plan de negocio → Producto → Operaciones → Actualización → Retiro). Dentro de él ocurren **uno o varios proyectos**, cada uno con su fase inicial, intermedia y final. El proyecto dura lo que dura el desarrollo; el producto dura hasta que se deja de usar.



El ciclo de vida del producto contiene a los ciclos de vida de sus proyectos.

Tipo	Cómo avanza	Ejemplos
Secuencial	Fases en orden lineal; una no empieza hasta que termina la anterior (revisión al final de cada fase). Generalmente no hay retorno. Dirigido por documentos	Cascada pura y sus variantes
Iterativo / incremental	Iteraciones sucesivas; cada una entrega un incremento de sw funcionando. Especificación, desarrollo y validación entrelazados.	Entrega por etapas, prototipado evolutivo; lo usa Ágil (y PUD como proceso definido)
Recursoivo	Se repite en vueltas, gestionando riesgos en cada una (mini-proyectos)	Espiral

Relación proceso de desarrollo – ciclo de vida: el proceso define *qué* actividades, roles y artefactos hay; el ciclo de vida define *en qué orden y cuánto* de cada actividad se hace. Un proceso **definido** admite cualquier ciclo de vida; un proceso **empírico** (ágil) solo el **iterativo-incremental**. El proceso es una implementación del ciclo de vida (abstracción) que tiene como objetivo construir un producto de sw.

Influencia en la administración de proyectos: elegir el ciclo de vida condiciona cómo se planifica, se controla y se mide el avance.

- **Secuencial:** se planifica todo al inicio; el progreso se ve en documentos; alcance fijo; un cambio impacta fuerte en tiempo y costo.
- **Iterativo:** se planifica por iteración; el progreso se ve en software funcionando; admite cambios.
- **Recursoivo:** la planificación gira en torno al análisis de riesgos de cada vuelta.

Ejemplo: si el análisis de riesgos dice que los requerimientos van a variar mucho, hay que elegir un ciclo iterativo, y eso cambia toda la gestión.

8. Ventajas y desventajas de cada ciclo de vida y criterios de elección

Los tres tipos básicos

Tipo	Ventajas	Desventajas
Secuencial	Planificación por adelantado (menos gastos de planificación); un método de gestión común; la documentación muestra el progreso; permite encontrar errores temprano en cada fase; da estructura a equipos poco experimentados	Los proyectos rara vez son secuenciales; exige requerimientos completos al inicio (no maneja incertidumbre); el cliente ve software funcionando muy tarde; un defecto tardío obliga a gran rediseño; equipos dependientes entre sí; documentación burocrática
Iterativo / incremental	Menor costo de adaptarse a cambios; el cliente participa y da feedback en cada incremento; entrega temprana de valor (lo	Desde la gestión: el proceso es menos visible; es caro documentar cada incremento; la

	crítico primero); ideal para requerimientos cambiantes	estructura del software tiende a degradarse con muchos incrementos
Recursivo (espiral)	Gestiona riesgos en sistemas grandes y complejos; evalúa factibilidad; genera productos reutilizables; a más inversión, menos riesgo	Identificar riesgos puede costar más que desarrollar; difícil definir hitos para pasar de vuelta; reutilizar puede salir caro; versión completa recién al final

Modelos de McConnell (Rapid Development, cap. 7) (no creo q entre)

Modelo	Idea central
Code and Fix	Programar y corregir sin especificación ni diseño
Cascada pura	Fases ordenadas, sin solapamiento, revisión al final de cada una, conducida por documentos
Cascada con fases solapadas	Las fases se superponen
Cascada con subproyectos	Cascada hasta el diseño arquitectónico, luego se divide en subproyectos
Cascada con retroalimentación	Se puede volver a una fase anterior
Espiral	Mini-proyectos; en cada vuelta: objetivos → riesgos → desarrollar y probar → planificar la siguiente
Entrega por etapas	Se entrega por etapas funcionales
Prototipado evolutivo	El concepto del sistema se desarrolla a medida que se avanza
Entrega evolutiva	Se fija tiempo o alcance y el otro varía
Diseño por planificación (cronograma)	Se asegura una entrega en la fecha, con lo más importante
Diseño por herramientas	Solo se incluye lo que soportan las herramientas existentes

Criterios para elegir un ciclo de vida

Se evalúa: nivel de **riesgo** e **incertidumbre**, **claridad y estabilidad de los requerimientos**, **conocimiento técnico** del equipo, **plazos**, posibilidad de **entregas parciales** y de **validar con el cliente**, y la **cultura** de la organización.

Ciclo de vida	Procesos que lo admiten	Elegirlo cuando...
Secuencial	Definidos	Alta certeza, requerimientos claros y exhaustivos; organización tradicional; no hay entregas parciales; el cliente no necesita el producto rápido
Iterativo / incremental	Definidos o empíricos	Hay incertidumbre y volatilidad de requerimientos; se pueden hacer entregas parciales; el cliente quiere lo principal cuanto antes; etapas tempranas del producto
Recursivo	Definidos	Sistemas grandes y complejos donde se necesita mucho control de riesgos en cada vuelta; no hay entregas parciales

9. Componentes de un Proyecto de Sistemas de Información.

¿Qué es un Proyecto de Software?

Esfuerzo temporal para crear un producto, servicio o resultado único. Características:

1. **Únicos:** por más parecidos que sean, algo los hace distintos (calendario, presupuesto, gente).
2. **Duración limitada:** son temporarios; al alcanzar el objetivo, terminan. **Una línea de producción no es un proyecto.**
3. **Orientado a objetivos:** están dirigidos a obtener resultados y ello se refleja a través de objetivos.
 - **No ambiguos:** Deben ser lo suficientemente claros para guiar al equipo.
 - **Medibles:** Permiten determinar el nivel de avance real.
 - **Alcanzables:** Deben ser factibles de realizar por el equipo asignado.
4. **Tareas Interrelacionadas con Esfuerzo y Recursos:** Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

“Planning is everything. Plans are nothing.” — Helmuth von Moltke. Lo valioso es el acto de planificar, no el documento.

Administración de Proyectos de Software

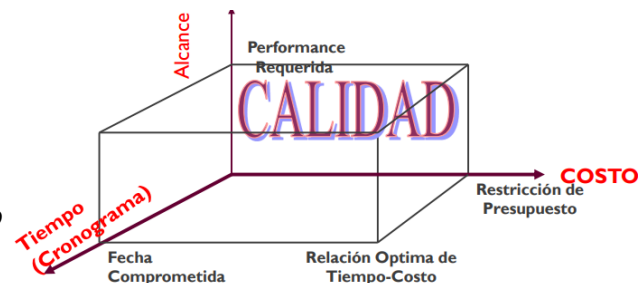
“Tener el trabajo hecho” en tiempo, con el presupuesto acordado y cumpliendo los requerimientos. Formalmente: aplicar conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto. Incluye: **identificar requerimientos, establecer objetivos claros y alcanzables y adaptar especificaciones, planes y enfoque a los distintos stakeholders**. Se gestiona porque **el software es complejo**. Dos grandes actividades: **planificación y monitoreo y control**.

Dificultades propias del software: el producto es intangible (cuesta medir avance), los proyectos son excepcionales (lo aprendido se extrapola poco) y los procesos varían según la organización.

La Triple Restricción (Alcance, Tiempo y Costo)

- **Alcance / objetivos:** ¿qué trata de alcanzar?
- **Tiempo:** ¿cuánto debería llevar?
- **Costo:** ¿cuánto debería costar?

El balance de los tres afecta directamente la **calidad** (en el centro del triángulo). “Un proyecto de alta calidad entrega lo requerido, satisfaciendo los objetivos en el tiempo y con el presupuesto planificados.”



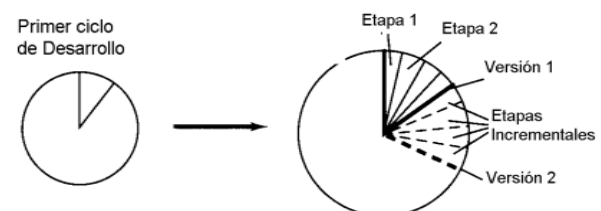
Balancearlos es responsabilidad del líder de proyecto. Se pueden fijar dos y la tercera se ajusta; en el enfoque tradicional se fija el **alcance** y de él se derivan tiempo y costo. En metodologías ágiles, el **tiempo** y **recursos** son fijos (iteraciones) y se gestiona un alcance variable y adaptable. **La calidad no se negocia.**

El Desarrollo de Software como Proceso Incremental: Cada nueva versión del producto software se desarrolla **incrementalmente** en una serie de pasos o etapas continuas.

Roles: líder, equipo y stakeholders

El **líder de proyecto** se relaciona con el cliente, la alta gerencia, subcontratistas, gerentes funcionales, organizaciones regulatorias y el equipo. En el enfoque tradicional decide y asigna tareas. Aptitudes: motivación, organización, innovación, empatía, comunicación, liderazgo, resolución de problemas. Responsabilidades: definir alcance, identificar involucrados y recursos, estimar, gestionar riesgos, controlar hitos, administrar cambios y reportar estado.

Equipo de proyecto: grupo de personas comprometidas con un conjunto de objetivos de los que se sienten mutuamente responsables. Diversos conocimientos y habilidades; trabajan juntos con sinergia; usualmente grupo pequeño; responsables como unidad.



Stakeholders: Todos los interesados o involucrados que tienen algo que decir o aportar respecto al producto o proyecto (equipo, dirección, líder, cliente, patrocinador).

El plan de proyecto

“Un plan es a un proyecto, lo que una hoja de ruta a un viaje.” Documenta: **¿qué** hacemos? **¿cuándo?** **¿cómo?** **¿quién?**

La planificación implica (8 actividades):

1. **Definición del alcance.** *Alcance del producto:* todas las características del producto; se mide contra la **Especificación de Requerimientos (ERS)**. *Alcance del proyecto:* todo el trabajo **y solo el trabajo** necesario para entregar ese producto; se mide contra el **Plan de Proyecto**.
2. **Definición de proceso y ciclo de vida** (el cómo; qué trabajo en cada fase, quién, cómo se aprueba).
3. **Estimación:** tamaño → esfuerzo → calendario → costo → recursos críticos (ver sección 10).
4. **Gestión de riesgos.**
5. **Asignación de recursos** (roles y personas).
6. **Programación** (cronograma detallado, ej. Gantt).
7. **Definición de controles.**
8. **Definición de métricas.**

Riesgos “”

Riesgo: un problema esperando para suceder; un evento que podría comprometer el éxito del proyecto.

El riesgo tiene dos variables: de impacto y la de la probabilidad de ocurrencia. **Exposición = probabilidad × impacto.**

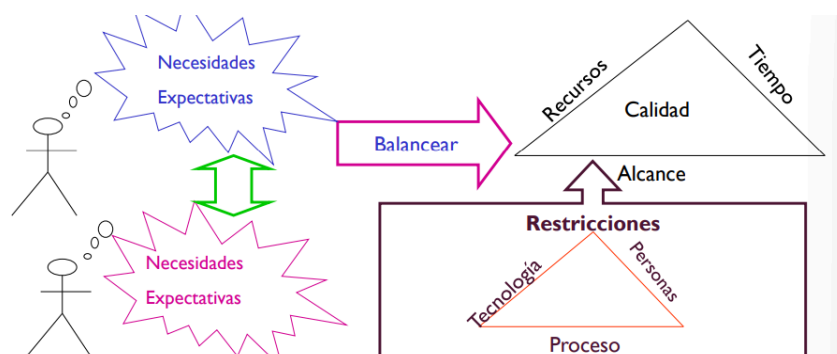
La lista se va actualizando. Los riesgos pueden tener un impacto hoy y otro distinto en el futuro.

- Tipos: **de proyecto** (calendario/recursos, ej. renuncia un diseñador clave), **de producto** (calidad/rendimiento, ej. componente comprado que falla) y **de negocio** (la organización, ej. un competidor lanza algo).
- Actitudes: negación, gestión reactiva o **proactiva** (planes de mitigación y contingencia).
- **Proceso de gestión de riesgos:** Identificar → Analizar → Planificar → Seguimiento y control → Aprendizaje (base de conocimiento de riesgos). Se hace **a lo largo de todas las fases**.

Métricas de software “”

- Tres dominios: **de proceso, de proyecto y de producto**. Las métricas de proyecto se consolidan para crear métricas de proceso públicas para toda la organización.
- **Métricas básicas:** tamaño del producto, esfuerzo, tiempo (calendario) y defectos.
- “El sueño del pibe”: el desarrollador, el equipo y la organización quieren medir cosas distintas (esfuerzo por tarea y defectos; tamaño, hitos y volatilidad de requerimientos; calendario, presupuesto y precisión de estimaciones).
- **Manténgalo simple:** “si estás a millas de tu destino, no tiene sentido medir en milímetros”. Preguntar: ¿da más información?, ¿es de beneficio práctico?, ¿dice lo que queremos saber?

Las métricas sirven para **balancear** necesidades y expectativas de los interesados contra la calidad y las restricciones (alcance, recursos, tiempo; tecnología, personas, proceso).



Monitoreo y control “”

Consiste en **comparar lo planificado con lo real** y corregir desvíos. **Tres factores top para el éxito:** monitoreo y feedback, una misión u objetivo claro, y comunicación.

Causas de fracaso en proyectos: fallas al definir el problema; planificar con datos insuficientes; la planificación la hizo “el grupo de planificaciones”; no hay seguimiento del plan; plan pobre en detalles; planificación de recursos inadecuada; estimaciones basadas en supuestos sin datos históricos; nadie estaba a cargo.

Preguntas de repaso de la diapositiva:

- **¿para qué se usan estimaciones, métricas y riesgos?** Las **estimaciones** predicen tamaño, esfuerzo, tiempo y costo, y son la base del plan (no son compromisos). Las **métricas** muestran el estado real para compararlo con lo planificado y alimentan futuras estimaciones. Los **riesgos** se gestionan para anticiparse a problemas con planes de mitigación y contingencia.
- **¿en qué momentos del proyecto?** Durante todo el proyecto. En la **planificación** se estima, se definen las métricas y se identifican los riesgos. En la **ejecución** (monitoreo y control) se reestima, se miden los resultados y se sigue cada riesgo. Al **cierre**, todo queda como dato histórico.
- **¿cómo se corrigen desviaciones?** Comparando lo planificado con lo real, buscando la causa y ajustando la triple restricción: reducir alcance, mover la fecha o ajustar recursos (ojo con la Ley de Brooks), sin bajar la calidad. Si el desvío vino de un riesgo, se ejecuta la contingencia. Después se replanifica y se comunica. En ágil el tiempo es fijo, así que se ajusta el alcance.

10. Estimaciones de software (en ambos enfoques: T y A)

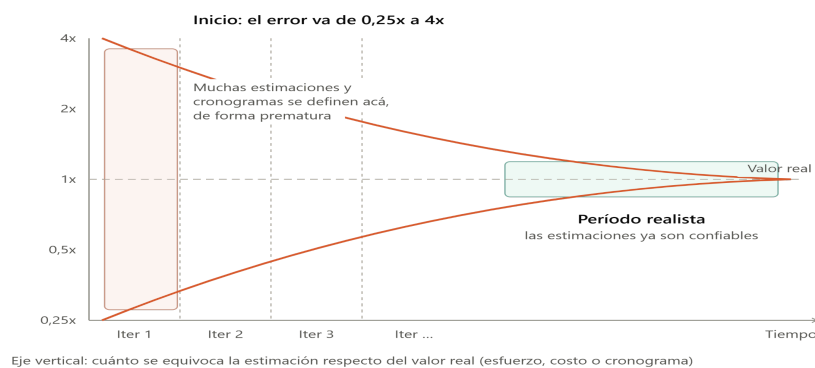
“La predicción es muy difícil, especialmente acerca del futuro.” — Niels Bohr

Estimar = obtener una aproximación de una medida para **predecir la completitud y administrar riesgos** (McConnell: predicción de cuánto durará o costará un proyecto).

Consideraciones clave:

- Por definición, una estimación **no es precisa**.
- **Estimar no es planear y planear no es estimar**.
- Las estimaciones son la **base** de los planes, pero los planes no tienen que ser iguales a lo estimado.
- **A mayor diferencia entre lo estimado y lo planeado, mayor riesgo**.
- **Las estimaciones no son compromisos**.

Cono de la incertidumbre: al inicio las estimaciones de esfuerzo, costo o cronograma pueden error entre **0,25x y 4x** (hasta 400%). Muchas se definen prematuramente en ese momento; el período realista para confiar llega con las iteraciones, cuando el error converge a 1x. Por eso hay que reestimar a medida que se avanza.



Qué se estima y en qué orden: tamaño → esfuerzo

(horas-persona lineales; ~80% del costo) → **calendario** → **costo** → **recursos críticos**. Error típico: estimar costo y tiempo sin haber estimado el tamaño.

De dónde vienen los errores de estimación: información imprecisa sobre el software o la capacidad del equipo; proyecto caótico o mal definido; imprecisión del propio método; y sobre todo **actividades omitidas**:

- Requerimientos faltantes.
- Actividades de desarrollo faltantes: documentación técnica, revisiones, datos de testing, mantenimiento de versiones previas.
- Actividades generales: enfermedad, licencias, cursos, reuniones de la empresa.
- Uso de buffers.

“Nunca tema que las estimaciones de los desarrolladores sean demasiado pesimistas: siempre generan cronogramas demasiado optimistas.” — Chris Peters (Microsoft)

Técnica fundamental: contar (antes que calcular o juzgar).

Métodos utilizados

Método	En qué consiste
Basados en la experiencia	Datos históricos, juicio experto (puro o Wideband Delphi) y analogía
Basados exclusivamente en los recursos	Se estima según la gente y el tiempo disponibles (“el trabajo se expande hasta consumir los recursos”)
Basado exclusivamente en el mercado	Se estima según lo que el cliente puede pagar o para ganarle a la competencia
Basados en los componentes del producto o en el proceso	Bottom-up (descomponer en unidades chicas y sumar) o top-down (bloques grandes)
Algorítmicos	Fórmulas con tamaño, complejidad, etc. y factores de ajuste; igual pueden fallar

Se recomienda usar varios métodos y contrastarlos → evita errores de estimación

Datos históricos: comparar datos del proyecto con uno anterior. Pueden ser de la industria, de la organización (proyectos cerrados) o del propio proyecto (fases anteriores). ¿Qué datos necesito? Los de las métricas básicas: **tamaño, defectos, calendario y esfuerzo**. Permiten que el conocimiento sea de la organización y no de una persona.

Juicio experto puro: un experto estudia las especificaciones y estima con su conocimiento. Es el enfoque más usado (~75% de las organizaciones). Problema: **si desaparece el experto, la empresa deja de estimar**. Para estructurarlo: tareas con granularidad aceptable, un checklist para asegurar cobertura y la fórmula optimista-habitual-pesimista:

$$E = \frac{O+4H+P}{6}$$

(“Un buen comienzo, pero un pésimo final”: no confiar solo en esto.)

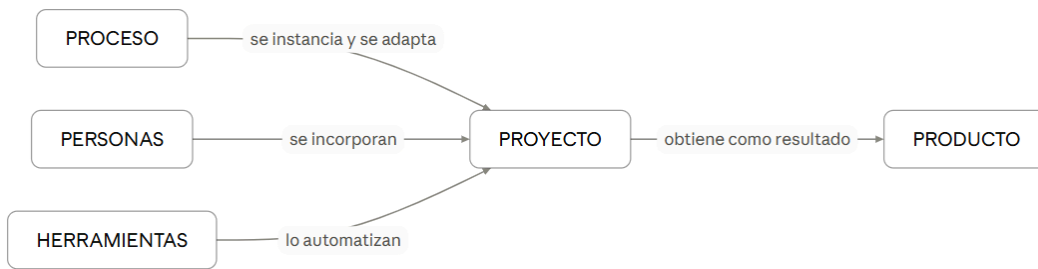
Wideband Delphi (juicio experto grupal):

1. Se dan las especificaciones a un grupo de expertos.
2. Se reúnen para discutir el producto y la estimación.
3. Remiten sus estimaciones individuales al coordinador.
4. Cada estimador recibe su estimación y las ajenas, de forma anónima.
5. Se reúnen de nuevo para discutir las.
6. Cada uno revisa la suya y la reenvía; se repite hasta converger razonablemente.

Las estimaciones en grupo suelen ser mejores que las individuales. Es la base del Planning Poker.

Problemas de las estimaciones tradicionales: estimar demasiado pronto, resistencia a reestimar, buscar precisión, que estime alguien distinto a quien hace el trabajo, y tratarlas como absolutas.

11. Vínculo proceso – proyecto – producto



En procesos definidos (el esquema del diagrama): el proceso (definición teórica de qué hacer para transformar requerimientos en software) **se instancia y se adapta** en un proyecto. Al proyecto **se incorporan personas**, el proceso **se automatiza con herramientas** y el proyecto **obtiene como resultado un producto**.

En procesos empíricos cambia el punto de partida: primero están las **personas con sus roles** (el equipo autoorganizado), y el proceso **no se adapta de uno predefinido, sino que el equipo lo crea** a partir de su experiencia, ajustándolo en cada ciclo de inspección y adaptación.

- **Proceso:** marco con actividades, roles y artefactos. No existe un proceso ideal. En el definido se toma solo lo que el proyecto necesita (adaptación); en el empírico emerge del equipo.
- **Proyecto:** unidad organizativa temporal donde se aplica el proceso con personas y recursos concretos. Acá actúan las disciplinas de gestión (planificación, monitoreo y control).
- **Producto:** el software resultante. Su ciclo de vida es más largo que el del proyecto y puede atravesar varios proyectos. Las disciplinas de soporte (SCM, calidad) cuidan su integridad.
- **Personas:** lo más importante, porque el software es una actividad humano-intensiva. El mejor proceso y el mejor plan no sirven sin un equipo capacitado, y en el enfoque empírico son directamente el centro.
- **Herramientas:** automatizan o facilitan las actividades del proceso.

Unidad 2 – Gestión Lean-Agile de productos de software

12. Base: filosofía ágil, manifiesto, empirismo y Lean

¿Qué es Ágil? No es una metodología ni un proceso: es una **ideología** con un conjunto definido de valores y principios que guían el desarrollo del producto. Es “un compromiso útil entre nada de proceso y demasiado proceso” (Fowler, 2001). Los métodos ágiles son **adaptables** (no predictivos) y **orientados a la gente** (no al proceso). Ágil está basado en un **proceso de control empírico** (transparencia, inspección, adaptación). Este nuevo pensamiento surgió desde los equipos de desarrollo.

Valores de los equipos ágiles: planificación continua y multinivel; equipos facultados, autoorganizados y completos; entregas frecuentes, iterativas y priorizadas; prácticas de ingeniería disciplinadas; integración continua; testing concurrente (TDD/ATDD).

Algunos frameworks ágiles: Scrum, XP, Crystal, ATDD, ShapeUp (y Kanban en Lean).

¿Por qué ir a Ágil? Acelera la entrega de valor, facilita adaptarse a cambios de prioridad, alinea negocio y TI, reduce riesgos y costos, mejora la visibilidad y las relaciones del equipo. **Técnicas más usadas:** daily standup (83%), backlogs priorizados (82%), iteraciones cortas (79%), retrospectivas (74%), planificación de iteración (69%).

Ser ágil no es ser indisciplinado: ni “sin plan / sin arquitectura” (mal ágil) ni “plan y arquitectura grandes por adelantado” (proyecto tradicional): **plan y arquitectura aproximados y adaptables** (buen ágil). Hacer todo “por pedacitos” no es ser ágil.

Manifiesto Ágil (2001) – 4 valores

Valoramos más lo de la izquierda, sin dejar de valorar lo de la derecha:

1. **Individuos e interacciones** sobre procesos y herramientas.
2. **Software funcionando** sobre documentación extensiva (se documenta lo que agrega valor, no “no documentar”). No significa “no documentar”: siempre hace falta guardar información sobre el producto y sobre el proyecto que lo construye. Lo que plantea el enfoque ágil es generar esa documentación **cuando hace falta y solo la que aporta valor**, en lugar de producir documentos extensos por adelantado.
3. **Colaboración con el cliente** sobre negociación contractual: en el enfoque tradicional, la fricción aparece cuando cambian los requerimientos: con un contrato de alcance y plazo fijos, cada cambio obliga a renegociar y mueve la fecha de entrega. En ágil, el cliente trabaja con el equipo y los cambios se incorporan sin conflicto.
4. **Responder al cambio** sobre seguir un plan. No significa “no planificar”: en ágil lo más importante es el **acto de planificar**, más que el plan que resulta. Al planificar, el equipo piensa en los objetivos, los riesgos y el alcance, y eso lo prepara para adaptarse cuando algo cambia. El plan en sí es solo una guía que se va ajustando. Como dice la frase de Moltke: *“Planning is everything. Plans are nothing.”*

Los 12 principios

1. Nuestra mayor prioridad es **satisfacer al cliente** con entregas tempranas y continuas de software valioso.
2. **Aceptar que los requisitos cambien**, incluso en etapas tardías.
3. **Entregar software funcional frecuentemente** (de 2 semanas a 1 mes).
4. **Responsables de negocio y desarrolladores trabajan juntos** día a día durante todo el proyecto.
5. Proyectos en torno a **individuos motivados**.
6. El método más eficiente de comunicar es la **conversación cara a cara**.
7. **El software funcionando es la principal medida de progreso**.
8. Los procesos ágiles promueven el **desarrollo sostenible**.
9. La atención continua a la **excelencia técnica y al buen diseño** mejora la agilidad. La calidad no se negocia.
10. **La simplicidad es esencial** (maximizar el trabajo no hecho).
11. Las mejores arquitecturas, requisitos y diseños emergen de **equipos autoorganizados**.
12. A intervalos regulares el equipo **reflexiona y ajusta** su comportamiento.

Triángulo ágil: el triángulo de hierro (alcance, tiempo, costo) pasa a ser una restricción; lo que importa es el **valor** (calidad extrínseca, para el cliente) y la **calidad** (intrínseca, la que le ponemos). En la versión modificada, las **personas** van en el lugar más importante.

¿Cuándo aplicar Ágil? En el terreno de lo **complejo** (productos nuevos, certeza y entendimiento parcial). Lo simple (ej. mantenimiento) no lo necesita tanto; la investigación cae en la “anarquía”.

¿Dónde está la agilidad? (iceberg): arriba se ven las **prácticas** (TDD, standup, pairing → comportamientos); debajo, los **principios** (calidad, simplicidad, colaboración, diseño evolutivo, software funcionando → modelos mentales); y en la base, los **valores** (individuos, interacciones, valentía, respeto, transparencia, inspección, adaptación, feedback → creencias). La agilidad está en los valores.

Filosofía Lean

Nace en Toyota (Japón, años 40), antes que Ágil. Busca **maximizar el valor para el cliente con el mínimo de recursos**: todo lo que no agrega valor es **desperdicio**. Usa **Just in Time** (“analice cuando lo necesite, no antes”). Los Poppendieck lo adaptaron al software.

7 principios Lean:

1. **Eliminar desperdicios.**
2. **Amplificar el aprendizaje.**
3. **Embeber la integridad conceptual** (calidad incorporada).
4. **Diferir compromisos** hasta el último momento responsable.
5. **Dar poder al equipo.**
6. **Ver el todo.**
7. **Entregar lo antes posible.**

7 desperdicios en software (entre paréntesis, manufactura): características extra (sobreproducción), trabajo a medias (inventario), proceso extra, búsqueda de información (se ataca con SCM), defectos, esperas (logística) y cambio de tareas/multitasking (se limita el WIP en Kanban).

Relación Lean-Ágil: Lean es previo; varios principios ágiles lo heredan. Ambos se orientan al cliente y al valor, a equipos motivados, a la adaptación al cambio, la calidad y la mejora continua.

13. Requerimientos ágiles

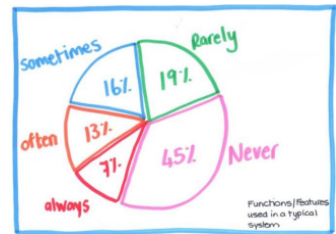
Idea base: lo más difícil es decidir **qué** construir (Brooks). En Ágil se empieza a construir con una **visión** del producto aunque no esté completamente definido, y los requerimientos emergen a lo largo del proyecto.

3 pilares de los requerimientos en Agile:

1. **Usar el “valor” para construir el producto correcto.**
2. **Determinar qué es “sólo lo suficiente”** (lo mínimo para empezar y generar feedback).
3. **Usar historias y modelos para mostrar qué construir** (junto con el cliente).

El costo del BRUF (Big Requirements Up Front): incluso los productos “exitosos” tienen mucho desperdicio. Según el Standish Group, de las funcionalidades: **45% nunca se usan**, 19% raramente, 16% a veces, 13% a menudo y solo **7% siempre**. Especificar todo al inicio genera funcionalidades que nadie usa.

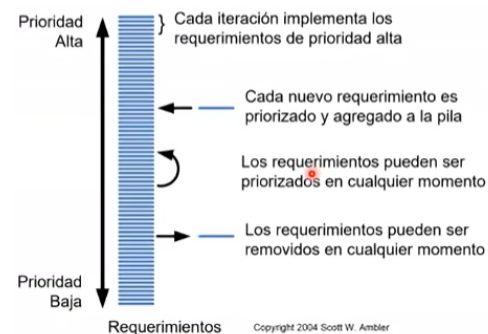
A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.



Gestión ágil de requerimientos (Product Backlog): “los requisitos cambiantes son una ventaja competitiva si podés actuar sobre ellos”.

El Product Owner mantiene los requerimientos ordenados por prioridad en una pila. Los requerimientos de prioridad baja se modelan con menor detalle y los de prioridad alta se detallan mucho más a medida que se acercan a su construcción.

- Cada iteración implementa los requerimientos de **mayor prioridad** (arriba de la pila, con más detalle).
- Cada nuevo requerimiento se **prioriza y agrega** a la pila.
- Se pueden **repriorizar o quitar en cualquier momento**.
- El backlog **nunca está al 100%**; lo prioriza el PO según valor de negocio.



Just in Time: “analice cuando lo necesite, no antes” (principios Lean: eliminar desperdicio y diferir compromisos).

Comunicación cara a cara: permite que fluya información vocal, subvocal y gestual con feedback rápido. La efectividad y la riqueza de la comunicación crece exponencialmente conforme a si se esta en un mismo espacio físico y con un pizarrón para construir el producto: papel → audio grabado → chat → video → teléfono → **2 personas en el pizarrón**. Los ágiles no escriben una ERS: lo compensan con la disponibilidad del cliente.

Tipos de requerimientos

Tipo	Dominio	Ejemplo
Requerimiento de negocio	Del problema	Disminuir un X% el tiempo en procesos manuales de atención al cliente
Requerimiento de usuario	Del problema	Consultar en línea el estado de cuenta de los clientes
Requerimiento funcional	De la solución	Generar reporte de saldos; recibir notificaciones por mail
Requerimiento no funcional	De la solución	Reporte en PDF; seguridad de credenciales según ley bancaria

Requerimiento de implementación	De la solución	Servidores en la nube
--	----------------	-----------------------

En gestión ágil se trabaja con requerimientos de **negocio y de usuario** (dominio del problema, lo que le interesa al cliente). **Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.**

Tradicional vs. Ágil:

	Tradicional	Ágil
Triple restricción	Alcance fijo; tiempo y costo se derivan	Tiempo y recursos fijos; el alcance varía (“cuánto software entrego en tanto tiempo con este equipo”)
Cuándo se relevan	Todos al principio (Up Front Specification)	Continuamente, justo a tiempo
Contenedor	ERS (persiste todo el ciclo de vida)	Product Backlog (transitorio; el detalle persiste en el producto y en las pruebas)
Fuente	Documento que el cliente lee y valida	Conversación cara a cara con el cliente/PO
Quién prioriza	A menudo el equipo	El cliente (Product Owner)

Por último (hay que asumir que): los cambios son la única constante; stakeholders: “no son todos los que están”; “el usuario dice lo que quiere cuando recibe lo que pidió”; no hay técnicas ni herramientas que sirvan para todos los casos; lo importante no es entregar una salida sino un **resultado de valor**.

Principios relacionados:

- **Ágiles:** 1 (satisfacer al cliente con releases tempranos y frecuentes), 2 (recibir cambios de requerimientos, aun en etapas finales), 4 (técnicos y no técnicos trabajando juntos TODO el proyecto), 6 (cara a cara), 11 (Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados).
- **Lean:** 1 (eliminar desperdicio), 4 (diferir compromisos), 6 (ver el todo).

14. User Stories

“Se las llama *stories* porque se supone que contás una historia. Lo que se escribe en la tarjeta no es importante; lo que hablás, sí.” — Jeff Patton

Término acuñado por Kent Beck (XP, 1999). Una US es una descripción corta de una funcionalidad valuada por un usuario. Es un requerimiento **de usuario** (no de software/sistema) y un **mecanismo para diferir una conversación**. La escribe (o prioriza) el **Product Owner**.

Son multipropósito. Una US es: una necesidad del usuario; una descripción del producto; un ítem de planificación; un token para una conversación; un mecanismo para diferir una conversación.

Valor: “la obtención de beneficio tangible o intangible” (Masa Maeda); “la utilidad, beneficio o satisfacción que le das al usuario por cada funcionalidad completa” (Pablo Lischinsky).

Las 3 partes (las 3 C)

1. **Tarjeta (Card):** la descripción breve, para planificar y como recordatorio.
2. **Conversación: la parte más importante;** el diálogo PO–equipo que compensa la falta de especificación. Es la “parte invisible”.
3. **Confirmación:** las pruebas de aceptación que demuestran que se implementó correctamente.

Sintaxis de la tarjeta:

Como <nombre del rol> **yo puedo** <actividad> **de forma tal que** <valor de negocio que recibo>

- **Rol:** quién realiza la acción o recibe el valor.
- **Actividad:** la acción que realizará el sistema.
- **Valor de negocio:** por qué es necesaria (lo más importante; permite priorizar).
- **Frase verbal** (opcional): título corto. Ej.: *Buscar destino por dirección* — “Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.”

Porciones verticales: cada US atraviesa todas las capas (GUI, lógica de negocio, base de datos) para entregar algo que funcione y tenga valor.

Modelado de roles

- **Tarjeta de rol de usuario:** describe un perfil genérico (ej. “Reclutador interno”: no experto en computadoras, usa la web con poca frecuencia pero intensamente; le importa recordar lo aprendido).
- **Técnicas adicionales:** **Personas** (una persona concreta e inventada: si los roles son clases, las personas son objetos; ej. “Mario”), **Personajes extremos** (diseñar una PDA para el Papa, una mujer con muchos novios o un traficante), **Usuarios representantes / proxies** (gerentes de usuarios, de desarrollo, marketing, vendedores, expertos del dominio, clientes, capacitadores y soporte).

Criterios de aceptación

Para qué sirven: definen **límites** para una US; ayudan a que el PO diga qué necesita para que tenga valor (requerimientos funcionales mínimos); dan al equipo una **visión compartida**; ayudan a derivar las **pruebas**; indican **cuándo parar** de agregar funcionalidad.

Buenos criterios: definen una **intención, no una solución** (“el usuario debe elegir al menos una cuenta para operar”); son **independientes de la implementación**; son relativamente de **alto nivel**. Ej.: “La altura de la calle es un número”; “La búsqueda no puede demorar más de 30 segundos”.

¿Y los detalles? (columna “Saldo”, formato 999.999.999,99, lista desplegable en vez de checkbox) van a la **documentación interna del equipo** o a **pruebas de aceptación automatizadas**.

Pruebas de aceptación

Expresan detalles resultantes de la conversación y complementan la US. Proceso de dos pasos: (1) identificarlas al dorso de la tarjeta; (2) diseñar las pruebas completas. Se prueban casos que **pasan** y que **fallan**. Ej. GPS: probar con país, ciudad, calle y altura existentes (pasa); calle inexistente (falla); altura inexistente (falla); país inexistente (falla); búsqueda de más de 30 s (falla). Ej. pago con tarjeta: Visa, MasterCard y AmEx (pasa), Diners (falla), tarjetas vencidas, montos mayores y menores de \$100. El PO acepta la US si todas se cumplen.

Definition of Ready y Definition of Done

- **DoR (Definición de Listo):** medida de calidad para determinar cuándo una US está en condiciones de **entrar** a una iteración. Mínimo: cumplir **INVEST**; el equipo y el PO pueden agregar condiciones.
- **DoD (Definición de Hecho):** checklist de cuándo una US está **terminada** (implementada, probada, integrada) para mostrarse al PO y poder ir a producción. Puede ser un estándar de la organización o definirla el equipo.

Modelo INVEST (Bill Wake)

Letra	Significa	Explicación
I	Independent	Calendarizables e implementables en cualquier orden
N	Negotiable	Describen el “qué”, no el “cómo”; los detalles se negocian
V	Valuable	Deben tener valor para el cliente/usuario (no para el equipo)
E	Estimable	Para ayudar al cliente a armar un ranking basado en costos
S	Small	Deben “consumirse” en una iteración

T	Testable	Se puede demostrar que fueron implementadas
---	----------	---

Una US puede no ser estimable por: incertidumbre de negocio (hablar con el PO), incertidumbre técnica (hacer un **spike**) o porque es muy grande (dividirla).

Niveles de abstracción

- **Épica:** una US demasiado grande para una iteración; se divide más adelante.
- **User Story:** cumple la DoR y entra en una iteración.
- **Tema:** conjunto de US relacionadas que se agrupan para estimar o planificar.
- En la diapositiva: Idea/Problema/Necesidad → Cambios en el negocio → Impacto en el cliente → Cambios en el software → Especificaciones.

Algo más sobre las US

- **No son especificaciones detalladas** (como los casos de uso): son expresiones de intención (“es necesario que haga algo como esto...”).
- No se detallan al inicio; necesitan poco o nulo mantenimiento y **pueden descartarse** después de implementarse; junto con el código sirven de entrada a la documentación.
- Diferir el análisis detallado lo más tarde posible (justo antes de empezar el trabajo). **Las US no son requerimientos de software.**
- Ventajas: comunicación verbal, comprensibles para todos, tamaño justo para planificar, permiten el desarrollo iterativo. Desventajas: en proyectos grandes cuesta ver la relación entre US; no escalan bien a equipos muy grandes.

¿Qué “huele mal”? No son US las que expresan restricciones o soluciones técnicas sin valor para el usuario: “el sistema corre en Windows XP y Linux”, “los gráficos se harán con una librería de terceros”, “se entrega el 30 de julio”, “desarrollado en JAVA”, “se usará Log4J”, “seleccionar el país desde una lista desplegable”. También huele mal una US que mezcla varias necesidades con “y” y criterios que describen la implementación.

Tips: palabras claras en los criterios; un paso a la vez (evitar “Y”); escribir desde la perspectiva del usuario; no olvidar la conversación; no forzar todo a formato de US.

Spikes

Tipo especial de historia para **quitar riesgo e incertidumbre** de una US u otra faceta del proyecto. Sirven para: familiarizarse con una tecnología o dominio; analizar una historia compleja para dividirla; ganar confianza frente a riesgos tecnológicos o funcionales.

Spike técnica	Spike funcional
Investiga enfoques en el dominio de la solución: performance, hacer o comprar, evaluar una tecnología	Incertidumbre sobre cómo el usuario interactuará con el sistema; se evalúa con prototipos y feedback

Ej.: “ver mi consumo de energía en un histograma” → spike técnico (tiempo de actualización, ancho de banda, push/pull) + spike funcional (prototipo del histograma para feedback).

Lineamientos: estimables, demostrables y aceptables; **la excepción, no la regla** (usarlos como última opción); implementarlos en una iteración **separada** de las historias resultantes, salvo que sean chicos.

15. Estimaciones ágiles

“No ser dogmático sobre nada. Ser pragmático es la clave del éxito en casi todo.”

Tips sobre las estimaciones:

- Si se usan como **compromisos** son muy peligrosas y perjudiciales.

- Lo más beneficioso es **el proceso de hacerlas**.
- Sirven como respuesta temprana sobre si el trabajo planificado es factible.
- Pueden ser una gran **protección para el equipo**.

Características: las features/stories se estiman con una medida de **tamaño relativo** llamada **story points (SP)**. Son medidas **relativas, no absolutas**, y **no están basadas en tiempo**. Estima **quien hace el trabajo** (el equipo de desarrollo, no el PO).

¿Por qué estimación relativa? Las personas no saben estimar en términos absolutos; somos buenos **comparando**; comparar es más rápido; genera mejor dinámica grupal y pensamiento de equipo; se aprovecha mejor el tiempo de análisis de las stories.

Tamaño: medida de la cantidad de trabajo necesaria para producir una feature/story. Indica cuán **compleja** es, cuánto **trabajo** requiere y cuán **grande** es.

Tamaño NO ES esfuerzo. El esfuerzo (horas-persona) depende de quien lo haga: habilidades, conocimiento, asunciones, experiencia, familiaridad con el dominio. Por eso las estimaciones basadas en tiempo son más propensas a error. Tampoco hay que confundir esfuerzo con calendario.

Escalas posibles (una vez elegida, no se cambia: “cambiaríamos el metro patrón”): números del 1 al 10; talles de remera (S, M, L, XL, XXL); serie 2^n (1, 2, 4, 8, 16, 32...); **Fibonacci** (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89...).

Story Point: unidad de medida específica **del equipo** que combina **complejidad, riesgo/incertidumbre (duda) y esfuerzo**; es a las stories lo que “el kilo” al peso. Da la idea del “peso” de cada story. La complejidad crece **exponencialmente**, por eso se usan series como Fibonacci. Lo importante es la consistencia dentro del equipo (una de 5 siempre menor que una de 8), no el número en sí.

Velocidad (velocity): métrica del progreso de un equipo. Se **calcula** (no se estima) sumando los SP de las US **completas** (DoD y aceptadas por el PO) en la iteración; las parcialmente terminadas **no cuentan** (gestión binaria). **La velocidad corrige los errores de estimación.** Se mira el promedio de varias iteraciones.

Duración de un release/proyecto = total de SP ÷ velocidad del equipo.

Reestimar solo cuando cambió la opinión sobre el tamaño **relativo** de una US; no porque el avance fue menor al esperado (eso lo equilibra la velocidad).

Planning Poker (Mike Cohn)

Combina **opinión de experto, analogía y desagregación**. Participan los **desarrolladores**: “las personas más competentes en resolver una tarea deben ser quienes la estiman”. El PO responde dudas pero no estima.

Cómo “decodificar” las cartas:

Carta	Significado
0	Quizás no tengas idea del producto o funcionalidad
1/2, 1	Funcionalidad pequeña (usualmente cosmética)
2–3	Pequeña a mediana. Es lo que queremos
5	Media. Es lo que queremos
8	Grande: se puede hacer, pero preguntarse si se puede dividir
13	¿Alguien puede explicar por qué no la podemos dividir?
20	¿Qué razón de negocio justifica semejante story?
40	No hay forma de hacerlo en un sprint

100	Confirmación de que algo está muy mal. Mejor ni arrancar
-----	--

Prerrequisitos: lista de stories a estimar y un mazo de cartas por estimador.

Pasos:

1. Determinar la **story base (canónica)** contra la que se comparan las demás (la más simple o conocida).
 - Se lee la story a todo el equipo.
 - Los estimadores la discuten y le preguntan al PO lo necesario.
 - Cada uno elige una carta y la pone boca abajo.
 - Se dan vuelta todas al mismo tiempo.
 - Si todos coinciden, ese es el valor. Si no, se discute, **empezando por el más alto y el más bajo**, y se vuelve a votar.
2. Se toma la próxima story y se discute con el PO.
3. Cada estimador le asigna un valor **por comparación con la story base** ("¿cuán grande, compleja o riesgosa es comparada con Z?") y se repite la votación.

Consideraciones: estimar considerando todo lo de la DoD (no solo programar); la estimación es time-boxed; estimar las menos US posibles (diferir compromisos).

Ágil es empírico: inspeccionar y adaptar es mandatorio.

Diferencias con la estimación tradicional:

	Tradicional	Ágil
Qué está fijo	El alcance (se estima tamaño y se derivan esfuerzo, tiempo y costo)	Tiempo y recursos; varía el alcance
Tipo	Absoluta (horas, LOC, puntos de función)	Relativa (story points)
Quién estima	Líder de proyecto o experto	El equipo que hace el trabajo
Precisión	Se busca precisión (base del plan)	La precisión es cara: se considera desperdicio
Compromiso	Se confunde con promesa	Es una probabilidad, no un compromiso
Cuándo	Al inicio, tras requerimientos y diseño, y cuando cambia el alcance	Al refinar el backlog / al planificar cada sprint

En **Lean** las estimaciones son opcionales y pueden ser desperdicio.

16. Gestión de producto

Idea central: el mayor riesgo no es fallar técnicamente, sino **hacer un producto que nadie use**. Construir más y más rápido **no es dar valor** (el 45% de las funcionalidades nunca se usa).

Valor vs. desperdicio

Para Lean, **valor es dar beneficios reales al cliente; todo lo demás es desperdicio**. La productividad no se mide en cuánto se construye por día, sino en **averiguar qué es lo correcto a construir**. El éxito es entregar algo que el cliente **realmente use**.

¿Qué es dar valor? Evolución de los productos



Dar valor ya no es solo satisfacer necesidades: también cuenta la **experiencia de uso**. Por la competencia, **lo viable ya no alcanza: hay que hacer algo memorable** que el usuario *quiera* usar.

Ciclo Lean Startup y la audacia del cero

Construir → Medir → Aprender y volver a empezar: se construye algo, se evalúa con datos reales y se ajusta lo planteado.

Audacia del cero: es más fácil conseguir inversión con cero clientes (invita a imaginar) que con pocos (generan dudas). Eso incentiva a **demorar el lanzamiento** hasta tener algo “perfecto”, lo que desperdicia esfuerzo y feedback. El MVP es el antídoto: salir cuanto antes.

MVP – Producto Mínimo Viable

Versión de un producto que el cliente percibe como **real** y permite obtener el **máximo aprendizaje validado con el mínimo esfuerzo** (Eric Ries). Sirve para saber si es lo que el cliente quiere **antes de invertir de más**.

- **Debe:** tener valor suficiente para que lo usen o compren; mostrar beneficio futuro para retener a los primeros usuarios; dar un ciclo de retroalimentación.
- **Parte de la UVP** (Propuesta de Valor Única): la hipótesis inicial de qué hace único al producto. El MVP existe para probarla, y **está bien hacer varios MVP** hasta validarla.
- **Se construye con fetas verticales:** un poco de cada capa (funcionalidad, fiabilidad, usabilidad, diseño), no solo la base funcional.
- **Qué entra:** lo **urgente e importante** (componentes críticos de la UVP). Lo importante no urgente se planea para después; lo urgente no importante se delega (APIs, terceros); lo demás se elimina.
- **Construirlo requiere juicio:** en caso de duda, simplificar; todo lo que exceda lo necesario es desperdicio.
- **Tipos:** desde una prueba de humo (sin producto, ej. un video) hasta un prototipo con funciones faltantes.
- **Errores comunes:** confundirlo con un MMF/MMP (el MVP busca aprender, no ganar); olvidar el “viable”; tratarlo como producto final.

Preparar un MVP: encontrar un nicho → roadmap realista → investigar la competencia → pre-vender → testear suposiciones → verificar que resuelve el problema correcto → focalizar en lo principal.

Supuestos de “salto de fe”

Son las suposiciones **más riesgosas**, de las que depende todo. El MVP las valida con dos hipótesis:

Hipótesis	Pregunta	Métrica
De valor	¿El producto entrega valor al usarlo?	Tasa de retención
De crecimiento	¿Cómo descubren el producto los nuevos usuarios?	Net Promoter Score (NPS) o referencias

Casos:

- **Facebook (tracción inicial, 2004):** más del 50% de los usuarios volvían todos los días (valor); en un mes casi el 75% del campus de Harvard lo usaba sin gastar en marketing (crecimiento). Con 150.000 usuarios y casi sin ingresos levantó USD 500.000 y luego 12,7 M. No validó factibilidad técnica: **validó comportamiento humano**.
- **Dropbox (prueba de humo):** riesgo técnico enorme (sincronización entre SO). Drew Houston hizo un **video de 3 minutos** mostrando cómo funcionaría, sin producto construido. La lista de espera beta saltó de 5.000 a 75.000 personas en una noche. **Lección: pensar en grande, empezar en pequeño.**

El dinosaurio Lean/Agile

1. **UVP:** se parte de una hipótesis de valor único.
2. **MVP:** se construye para probarla.
3. **Señal dispersa:** si cada cliente pide una “X” distinta, todavía no hay mercado.
4. **Pivote (MVP2):** si todos piden la misma X, se revisa la hipótesis y se hace un MVP2.

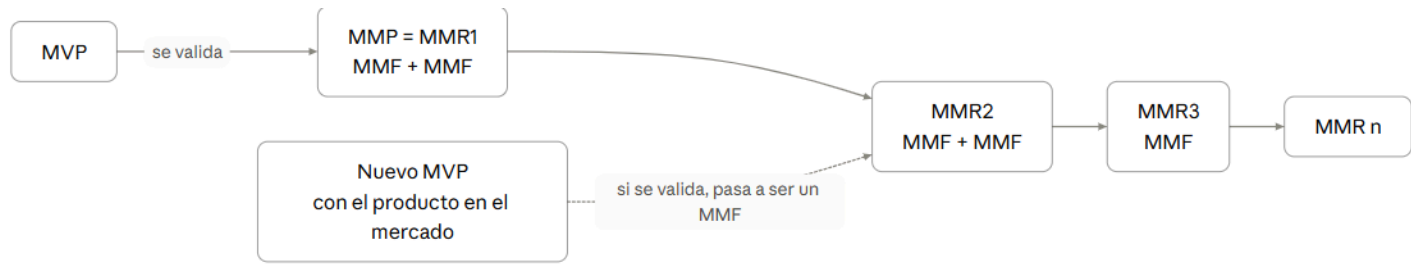
5. **Tracción:** con primeros usuarios, se buscan áreas para crecer.
6. **MMF:** donde hay **certeza de valor**, se construyen MMFs (piezas chicas, por time to market).
7. **MVF:** donde hay potencial pero **incertidumbre**, se prueba una característica “pionera” (un mini MVP). Si funciona, se hacen más MMFs ahí; si no, se cambia de enfoque.
8. **Modelo completo:** el producto crece combinando MMF y MVF. Como son grandes para desarrollar, se cortan en porciones finas (“**dinosaurio carpaccio**”): las **User Stories**.

Los “mínimos” después del MVP

Sigla	Qué es	Para qué
MVP (Producto Mínimo Viable)	Versión mínima de un producto nuevo, para un subconjunto de clientes; más cerca de un prototipo	Aprender
MVF (Característica Mínima Viable)	Mini MVP de una característica, probada con usuarios tolerantes. Busca encontrar aquella funcionalidad que hace la diferencia.	Explorar un área incierta
MMF (Característica Mínima Comercializable)	La pieza más chica de funcionalidad que se puede liberar con valor; es parte de un MMR/MMP. Validación del producto en términos de la comercialización	Aportar valor con certeza
MRF (Características Mínimas del Release)	Lo mínimo que debe tener un release para salir.	Definir cada release
MMP (Producto Mínimo Comercializable)	Primer release, para early adopters. MMP = MMR1	Salir al mercado
MMR (Release Mínimo Comercializable)	El incremento más chico que da valor nuevo (MMR2, MMR3...)	Hacer crecer el producto

Clave: MVP y MVF prueban **hipótesis** (aprender); MMF, MMP y MMR entregan valor con **certeza** (ganar).

Relación entre MVP, MMF, MMP y MMR (línea de tiempo)



Los releases (MMR) se arman con MMFs y el primero es el MMP. El MVP no se hace una sola vez: con el producto en el mercado se lanzan **nuevos MVP** para probar ideas, y si se validan pasan a ser MMFs de un release posterior.

17. Framework Scrum (Scrum 2026: la Guía Expandida)

Definición: Scrum es un **marco de trabajo liviano** que ayuda a personas, equipos y organizaciones a **generar valor** a través de **soluciones adaptativas para problemas complejos**. Se basa en el **empirismo** y el **pensamiento Lean**; es iterativo e incremental; deliberadamente incompleto: guía relaciones e interacciones en lugar de prescribir procesos detallados

Principios de Scrum: control de proceso empírico, autoorganización, colaboración, priorización basada en valor, time-boxing y desarrollo iterativo.

En pocas palabras: el PO fija el Product Goal, el Scrum Team autogestionado convierte el trabajo en valor en cada Sprint, se inspecciona el resultado con los stakeholders y se adapta, con el apoyo de los Supporters; y se repite.

El motor: empirismo y ciclo OODA. Los 3 pilares son **transparencia** (mostrar la realidad), **inspección** (detectar desvíos) y **adaptación** (ajustar enseguida; inspeccionar sin adaptar es desperdicio). En 2026 se suman los **valores** —**Coraje, Foco, Compromiso, Respeto y Apertura**— leídos con el ciclo **OODA**: apertura y respeto para **observar**, coraje para **orientarse**, foco para **decidir** (experimentos seguros para fallar) y compromiso para **actuar**. La confianza es la base: sin coraje para decir “no” y respeto para criticar ideas (no personas), el empirismo falla.

Roles y Scrum Team

Hay 4 roles, pero solo 3 tienen *accountability* (responsabilidad) y forman el Scrum Team: PO, Product Developers y Scrum Master.

Rol	Naturaleza	Responsabilidad principal
Product Owner	Humano	El qué y el por qué : maximizar el valor a largo plazo, gestionar el Product Backlog y la estrategia del producto. Es una persona, no un comité ni un escriba de tickets
Product Developer	Humano o IA	El cómo y la calidad : crear incrementos usables, cumplir la Definition of Output Done, autogestionar el Sprint Backlog. Mantra: “stop starting, start finishing”
Scrum Master	Humano	La efectividad : líder servicial y agente de cambio, remueve impedimentos organizacionales. No es secretario ni administrador de Jira
Stakeholder	Humano, entidad o IA	Interesados o afectados por el producto (clientes, usuarios, patrocinadores, legisladores). Dan feedback, requisitos y validan el valor
Supporter	Stakeholder específico	Líderes y gerentes que apoyan la adopción de Scrum y eliminan barreras. Si un manager no empodera al equipo, no es Supporter

El Scrum Team (“célula profesional”): pequeño (en general 10 o menos), sin sub-equipos ni jerarquías, **multifuncional** y **autogestionado** (decide quién, qué, cómo, cuándo y dónde); **profesionalismo** (responsabilidad “desde la cuna hasta la tumba” del producto) y **diversidad cognitiva**.

IA: puede integrarse como Product Developer, pero con **“human in the loop”**: la responsabilidad final siempre es humana, con uso ético y supervisado.

Liderazgo y cambio: el liderazgo es una conducta, no un cargo. *Genchi Genbutsu* (“ve, escucha y entiende”): decidir con hechos reales, no reportes distantes. El cambio es emergente: se guía una dirección, no se fuerza un destino.

Eventos/ ceremonias?

Evento	Qué es	Timebox
Sprint	Contenedor de los demás eventos; crea cadencia. Sprints cortos = menor costo de error	1 mes o menos
Sprint Planning	Define el por qué (Sprint Goal), el qué (ítems) y el cómo (plan). Planificar holgura (slack)	8 h máx.
Daily Scrum	Replanificación de los Developers hacia el Sprint Goal. No es un reporte al Scrum Master	15 min
Sprint Review	Sesión de trabajo con stakeholders, no una presentación: inspecciona el incremento y valida el valor (DoD Outcome) (¿resolvimos el problema del usuario?)	4 h máx.
Sprint Retrospective	Mejora del equipo, procesos, herramientas y DoD; las mejoras se aplican cuanto antes	3 h máx.
Refinamiento del PB	Actividad continua, no evento: descomponer ítems y reducir incertidumbre	~10% del sprint

Durante el Sprint, el **Sprint Goal es inamovible**; el alcance se puede negociar entre PO y Developers. **No hay “sprints de hardening”**: la calidad se construye en cada Sprint. Solo el PO puede cancelar un Sprint.

Artefactos y compromisos

Artefacto	Compromiso
Product Backlog: lista emergente y ordenada de lo que se necesita	Product Goal: objetivo estratégico a mediano/largo plazo (el “por qué” estratégico)
Sprint Backlog: Sprint Goal + ítems seleccionados + plan de los Developers para el Sprint	Sprint Goal: objetivo único y coherente del sprint (el “por qué” táctico)
Incremento: el “ladrillo” de valor entregado (output)	Definition of Output Done
Producto: lo que genera el valor (outcome)	Definition of Outcome Done

Product Thinking: los proyectos terminan, los productos evolucionan. Product Vision (norte, largo plazo) → Product Goal (paso concreto hacia la visión) → Product Backlog (ordenado por valor). Discovery y delivery ocurren dentro del Sprint.

Output vs. Outcome: el cambio fundamental

	DoR (Definition of Ready)	DoD Output	DoD Outcome
Estatus	Cualidad emergente del refinamiento, no un compromiso	Compromiso formal	Compromiso formal
Aplica a	Product Backlog Item	Incremento	Producto
Pregunta	¿Está claro y es chico como para empezar?	¿Lo construimos bien? (calidad técnica)	¿Construimos lo correcto? ¿Generó valor?
Criterios	Sin ambigüedades, completable en pocos días, spikes resueltos	Código limpio y testeado, integrado, cumple estándares	Cambio en el comportamiento del usuario, impacto en el negocio
Cuándo	Antes de entrar al Sprint	Dentro del Sprint; nunca se debilita por presión de tiempo	A menudo después del lanzamiento (telemetría)
Responsable	Scrum Team (los Devs estiman)	Developers	Product Owner (última palabra)

“El output es el requisito previo; el outcome es el éxito real.” Riesgo crítico: confundir “entregar el output” con “generar el outcome”.

Scrum 2020 vs. Scrum 2026 (SGEP): qué cambió

La diapositiva lo presenta como una evolución: 2020 era la **base** (elementos esenciales, riesgo de volverse una “fábrica de funcionalidades”); 2026 es la **expansión** (roles, outcomes, IA, adaptación y refinamiento).

Eje	Scrum 2020	Scrum 2026 (SGEP)
1. Roles	Solo 3 accountabilities: Scrum Master, PO y Developers. “Stakeholder” se menciona pero no se define	Vuelve el concepto de roles (4: PO, Product Developer, SM y Stakeholder). Aparece el Supporter. PO y SM deben ser humanos

Developers	Developers (personas)	Product Developers: humanos o IA (al menos uno humano)
2. Compromisos y calidad	Una sola Definition of Done , compromiso de los Developers	Se divide en DoD Output (calidad técnica del incremento) y DoD Outcome (valor real del producto, responsabilidad del PO)
Artefactos	Product Backlog, Sprint Backlog e Incremento	Se suma el Producto como artefacto (compromiso: DoD Outcome)
Sprint Goal	Lo crean y son dueños los Developers	Propiedad colectiva de todo el Scrum Team
3. IA	No se menciona	Socio, no reemplazo: miembro del equipo que potencia el refinamiento, detecta ambigüedades y valida hipótesis; mejora el control empírico con analíticas; siempre con human in the loop
4. Teoría	Empirismo y Lean	Se agregan OODA (decisiones rápidas con incertidumbre), sistemas adaptativos complejos (el equipo como sistema que evoluciona) y Product Thinking (ciclo de vida completo) en vez de Project Thinking (proyectos que dejan productos “huérfanos”)
5. Prácticas y planificación	Refinamiento como actividad secundaria; solo Product Goal	Refinamiento crítico y continuo para reducir riesgo; se incluye la Product Vision (norte a largo plazo) con el Product Goal como paso intermedio; planificación progresiva (just-in-time): planificar solo lo necesario y dejar que el plan emerja en el Sprint, con holgura
Foco general	Entregar incrementos (output)	Entregar valor comprobado (outcome) a largo plazo

18. Planificación en Scrum (release y sprint), capacidad, herramientas y métricas ágiles

Múltiples niveles de planificación (“cebolla”)

Estrategia → **Portfolio** → **Producto** → **Release** → **Iteración** → **Día**.

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y POs	Administrar un portfolio de productos	Backlog de portfolio
Producto (visión)	Varios meses o más	PO y stakeholders	Visión y evolución del producto	Visión, roadmap y características
Release	3 (o menos) a 9 meses	Scrum Team y stakeholders	Balancear valor y calidad con alcance, cronograma y presupuesto	Plan de release
Iteración (sprint)	1 semana a 1 mes	Scrum Team	Qué entregar en el próximo sprint	Sprint Goal y Sprint Backlog
Día	Diaria	Developers (y SM)	Cómo completar lo comprometido	Inspección del progreso

¿Qué y cuándo estimar? Portfolio backlog → productos en **talles** (S, M, L, XL); Product backlog → features en **story points**; Sprint backlog → tareas en **horas** ideales.

Niveles de granularidad: meses (más grande que un release, épica) → semanas (más grande que un sprint) → días (listo para un sprint, US) → horas (tareas).

Planificación de portfolio: envisioning → filtro económico (go / no-go) → portfolio planning. **Planificación de producto:** idea → filtro estratégico (go / no-go) → envisioning (entradas: idea inicial o pivotada, horizonte, fecha, presupuesto, umbral de confianza; salidas: **visión, Product Backlog y roadmap**) → release planning.

Planificación de release

Participan stakeholders internos y el Scrum Team. Entradas: visión, Product Backlog, roadmap y **velocidad**. Actividades: revisar restricciones (alcance, fecha, presupuesto), grooming del backlog, revisar MRFs y mapear sprints. Salida: **plan de release** (rango de features a costo fijo o rango de sprints/costos), MRFs y sprint map.

Ejemplo de la diapositiva: con **velocidad promedio = 20 SP**, se van tomando features del backlog ordenado (A=5, B=3, C=2, D=8, E=2 → sprint 1 = 20) y así se reparten en sprints 1, 2, 3 y 4–7. Un release agrupa varios sprints (release 0.5, 1.0, X.Y).

Cadencia de los sprints: release luego de múltiples sprints, luego de cada sprint, o luego de cada feature.

Planificación de iteración y diaria

- **Sprint Planning** (primera actividad de cada sprint): del Product Backlog refinado se elige **qué hacer** (características) y en el Sprint Backlog se descompone en **tareas = cómo hacerlo**.
- **Daily Scrum:** cada 24 horas, inspección y adaptación durante la ejecución del sprint.

Capacidad del equipo en un sprint

Capacidad total del sprint = trabajo en los PBIs + **buffer del sprint** + tiempo fuera del personal + otros compromisos externos (soporte, mantenimiento, otros proyectos) + otras actividades del sprint (planning, review, retrospective, grooming). Solo lo primero se compromete en ítems. **Se estima y se mide**; equipos maduros en SP, menos maduros en horas ideales.

$$\text{Capacidad} = WH \times DA$$

WH = horas de trabajo disponibles por día (ideales); DA = días disponibles en la iteración.

Ejemplo por persona (diapositiva):

Persona	Días disponibles (sin tiempo personal)	Días para actividades Scrum	Horas por día	Horas de esfuerzo disponibles
Jorge	10	2	4–7	32–56
Betty	8	2	5–6	30–36
Simón	8	2	4–6	24–36
Pedro	9	2	2–3	14–21
Raúl	10	2	5–6	40–48
Total				140–197

Cálculo: (días disponibles – días Scrum) × horas por día. Ej. Jorge: $(10 - 2) \times 4 = 32$ y $(10 - 2) \times 7 = 56$.

Ejemplo por sprint: equipo de 8, 6 h/día, sprints de 10 días; solo 4 miembros disponibles en los sprints 1 y 2; uno se casa en los sprints 5 y 6.

Sprint	1	2	3	4	5	6	7	Total
Horas	240	240	480	480	420	420	480	2.760
Story points	30	30	45	60	58	52	60	335

$(4 \times 6 \times 10 = 240; 8 \times 6 \times 10 = 480; 7 \times 6 \times 10 = 420.)$

Ambiente de trabajo: abierto (el silencio absoluto es mal signo), pizarrones, el equipo define sus horarios.

Herramientas de Scrum

- **Tarjeta de tarea:** código del ítem del backlog, nombre de la actividad, iniciales de la persona, número de día del sprint y tiempo estimado en horas.
- **Taskboard:** columnas **Story | To Do | In Process | To Verify | Done**. Buena granularidad: muchas tareas chicas; mala granularidad: una sola tarea por story (no se ve el avance).
- **Sprint Burndown Chart:** trabajo restante (SP u horas) vs. días del sprint, contra una línea ideal. Curva por encima de la ideal = **atrasado**; por debajo = **adelantado**.
- **Seguimiento del producto:** **Release Burnup** (trabajo completado acumulado vs. alcance total por sprint; muestra cambios de alcance), **Product Burndown** y gráfico de SP entregados por sprint.

Métricas en ambientes ágiles

“No todo lo que puede contarse cuenta, y no todo lo que cuenta puede contarse.” (Einstein)

Regla de oro: la medición es una salida, no una actividad. Filosofía minimalista: medir lo necesario y nada más. Dos principios guían la elección: “nuestra mayor prioridad es satisfacer al cliente mediante entregas tempranas y continuas de software valioso” y “el software trabajando es la principal medida de progreso”. Las métricas son del equipo y **no se extrapolan** a otros equipos.

Métrica	Qué mide	Cómo
Velocidad	Progreso del equipo (métrica de producto)	Suma de SP de las US completas y aceptadas en la iteración; se calcula, no se estima; corrige errores de estimación
Capacidad	Compromiso de trabajo para un sprint (métrica de proyecto)	WH × DA; se estima al inicio del sprint (en horas ideales o SP)
Running Tested Features (RTF)	Cantidad de features testeadas y funcionando	Conteo por iteración; debe crecer. Curva plana o descendente = problema. No considera la complejidad de cada feature

Comparación de enfoques: tradicional mide esfuerzo, tiempo, costos y riesgos (proceso, proyecto y producto); ágil mide velocidad, capacidad y RTF (producto); Lean/Kanban mide lead time, cycle time, touch time y eficiencia del proceso (touch time / lead time). Las métricas de defectos se usan en todos.

Unidad N° 3: Software Configuration Management (SCM)

19. Gestión de Configuración de Software (SCM) – “más allá del commit/update”

El software es un conjunto de programas, procedimientos, reglas, documentación y datos: información estructurada con propiedades lógicas y funcionales, creada y mantenida en varias formas y representaciones. **Evoluciona** (requerimientos, diseño, fuentes y objetos de enero cambian en febrero...) y sin control aparece el problema de no saber qué versión de cada cosa corresponde a qué momento.

Los cambios tienen origen en: cambios del negocio y nuevos requerimientos; soporte de cambios de productos asociados; reorganización de prioridades por crecimiento; cambios en el presupuesto; defectos a corregir; oportunidades de mejora.

SCM como disciplina de soporte: actividad “paraguas”, **transversal a todo el proyecto** y relevante para el **producto a lo largo de su ciclo de vida**. Junto con el aseguramiento de calidad, el control de calidad de producto y de proceso y la prueba de software, forma las disciplinas de soporte. Origen: mediados de los años 50, cuando la CM de hardware y control de producción se aplicó al software.

Definición (ANSI/IEEE 828, 1990): disciplina que aplica **dirección y monitoreo administrativo y técnico** a: **identificar y documentar** las características funcionales y técnicas de los ítems de configuración, **controlar los cambios** de esas características, **registrar y reportar** los cambios y su estado de implementación y **verificar correspondencia con los requerimientos**.

Propósito: establecer y mantener la **integridad** de los productos de software a lo largo de su ciclo de vida. Involucra: identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.

Un producto tiene integridad cuando: satisface las necesidades del usuario; puede ser fácil y completamente **rastreado** durante su ciclo de vida; satisface criterios de performance; cumple sus expectativas de costo. (“El software: un blanco móvil.”)

Problemas en el manejo de componentes (lo que SCM evita): pérdida de un componente; pérdida de cambios (el componente que tengo no es el último); falta de sincronía fuente-objeto-ejecutable; regresión de fallas; doble mantenimiento; superposición de cambios; cambios no validados.

Conceptos clave

Concepto	Definición
Ítem de configuración (IC/SCI)	Todo artefacto del producto o del proyecto que puede cambiar o necesita compartirse, y del que necesitamos conocer estado y evolución. Ej.: plan de CM, propuestas de cambio, visión, riesgos, plan de desarrollo, prototipo de interfaz, guía de estilo, manual de usuario, requerimientos, plan de calidad, arquitectura, plan de integración, planes de iteración, estándares de codificación, casos de prueba, código fuente, gráficos e íconos, programa de instalación, documento de despliegue, formulario de aceptación
Versión	La forma particular de un artefacto en un instante o contexto dado. El control de versiones se refiere a la evolución de un único IC (o de cada IC por separado); se representa como grafo (evolución lineal)
Variante	Versión de un IC (o de la configuración) que evoluciona por separado ; representa configuraciones alternativas (ej. según plataforma máquina + SO, o funciones opcionales)
Configuración del software	Conjunto de ítems de configuración con su versión correspondiente en un momento determinado (una “foto”)
Repositorio	Contenedor de los ICs; mantiene la historia de cada IC con sus atributos y relaciones; se usa para evaluar el impacto de los cambios; puede ser una o varias bases de datos. Funciona con check out (extracción) y check in (devolución)
Línea base (baseline)	Configuración revisada formalmente y acordada ; sirve de base para desarrollos posteriores y solo cambia mediante un procedimiento formal de control de cambios . Permite volver atrás y reproducir el entorno en un momento dado. Se marca con etiquetas (tags) ; no confundir con la versión del producto
Rama (branch)	Bifurcación del desarrollo a partir de la rama principal (trunk/master); permiten experimentar; pueden tener semántica; se integran (merge) o se descartan

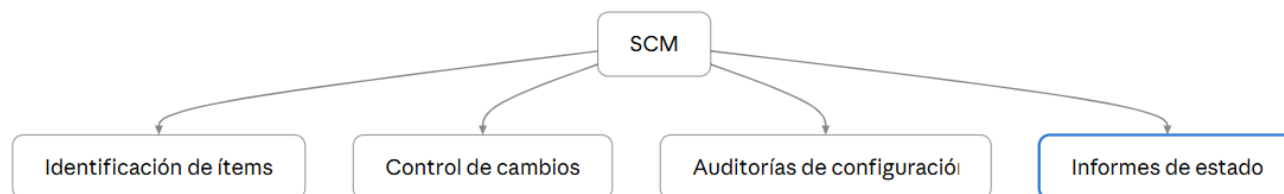
Repositorios:

Centralizado	Descentralizado
Un servidor contiene todos los archivos con sus versiones; los administradores tienen más control; si falla el servidor, “estamos al horno”	Cada cliente tiene una copia completa del repositorio; si un servidor falla es “copiar y pegar”; posibilita otros workflows

Tipos de líneas base: **de especificación** (requerimientos, diseño; no tienen código) y **de productos** que pasaron un control de calidad definido previamente (operacionales; la primera coincide con el primer release).

Integración de ramas: la operación se llama **merge**, lleva los cambios a la rama principal; pueden surgir **conflictos** (se resuelven con *diff*); toda rama debería eventualmente integrarse o descartarse.

Actividades fundamentales de la Gestión de Configuración de Software



1. Identificación de ítems de configuración: identificación unívoca de cada IC; convenciones y reglas de nombrado; definición de la estructura del repositorio; ubicación de cada IC dentro de ella. Según su ciclo de vida:

De producto	De proyecto	De iteración
ERS, arquitectura, código, manual de usuario (viven mientras exista el producto)	Plan de proyecto, cronograma (viven lo que el proyecto)	Plan de iteración, reporte de defectos (viven lo que la iteración)

2. Control de cambios: se origina en un **requerimiento de cambio** a uno o varios ICs que están en una **línea base**. Es un **procedimiento formal** con distintos actores y evaluación de impacto.

1. Necesidad de cambio → **propuesta/petición de cambio** sobre la línea base.
2. El **Comité de Control de Cambios (CCB)** hace el **análisis de impacto** (aspectos técnicos, efectos colaterales, costos, impacto general).
3. **Decisión** del comité: aceptación o rechazo (se informa al cliente).
4. Si se acepta: **orden de cambio de ingeniería** (descripción, limitaciones, criterios de revisión y auditoría); se ubica en la cola de cambios.
5. Desarrollo realiza el cambio; el comité hace la **revisión de partes** y se establece la nueva línea base.
6. **Notificación a las partes** interesadas.

Comité de Control de Cambios: representantes de todas las áreas involucradas (análisis, diseño, implementación, testing y otros interesados; también líder de proyecto o cliente según el cambio).

3. Auditorías de configuración: las hace alguien **independiente y objetivo**, sobre una **línea base**, y requieren un plan de SCM.

Auditoría física (PCA)	Auditoría funcional (FCA)
Asegura que lo indicado para cada IC en la línea base se alcanzó realmente: el producto construido es consistente con la documentación (repositorio, nombrado, ubicación). Hace verificación . Se hace primero	Evaluación independiente de que la funcionalidad y performance reales de cada IC sean consistentes con la especificación de requerimientos (matriz de rastreabilidad). Hace validación

La auditoría sirve a dos procesos: **Validación** = el problema se resuelve de manera apropiada, el usuario obtiene el producto correcto. **Verificación** = el producto cumple los objetivos definidos en la documentación de las líneas base; todas las funciones se ejecutan con éxito y los test cases están “ok” o constan como “problemas reportados” en la nota de release.

4. Registro e informe de estado: mantiene los registros de la evolución del sistema; maneja mucha información, por eso suele automatizarse; incluye reportes de **rastreabilidad** de todos los cambios a las líneas base. Responde preguntas como: ¿cuál es el estado del ítem?, ¿un requerimiento de cambio fue aprobado o rechazado por el CCB?, ¿qué versión de un ítem implementa un cambio aprobado?, ¿cuál es la diferencia entre dos versiones?

Plan de Gestión de Configuración

¡También se planifica (y temprano)! Debe incluir: **reglas de nombrado** de los ICs; **herramientas** para SCM; **roles e integrantes del comité**; **procedimiento formal de cambios**; **plantillas de formularios**; **procesos de auditoría** (Bonugli agrega estructura del repositorio y registros a mantener).

SCM en ambientes ágiles

- **Sirve a los practicantes** (el equipo) y no al revés.
- **Hace seguimiento y coordina** el desarrollo en lugar de controlar a los desarrolladores.
- **Responde a los cambios** en lugar de tratar de evitarlos.
- Busca ser transparente y “sin fricción”, **automatizando** tanto como sea posible.
- Coordinación y automatización frecuente y rápida.
- **Eliminar el desperdicio**: no agregar nada más que valor.
- Documentación Lean y trazabilidad.
- Feedback continuo y visible sobre calidad, estabilidad e integridad.

Tips: es responsabilidad de **todo el equipo**; automatizar lo más posible; educar al equipo; tareas de SCM **embebidas** en las demás tareas del sprint.

Para debatir:

- *¿Qué pasa con el Comité de Control de Cambios?* No existe como tal: el PO prioriza los cambios en el Product Backlog y el equipo autoorganizado los maneja.
- *¿Qué ítems de configuración tenemos?* Product Backlog, Sprint Backlog, US, código, casos de prueba, DoD, etc. Se mantienen repositorio y esquema de nombrado.
- *¿Qué pasa con las auditorías?* Es la única actividad que no encaja con Ágil puro (control externo vs. equipo autoorganizado); pueden obviarse.
- *¿Y los reportes de estado?* Se reemplazan por la transparencia del equipo y la automatización (tableros, herramientas).

Prácticas continuas:

Práctica	Qué es
Integración continua (CI)	Integrar el código al repositorio compartido varias veces al día; cada integración se verifica con pruebas automatizadas para detectar problemas temprano
Entrega continua (CD)	El software siempre está listo para liberarse a producción; se automatizan las pruebas de aceptación. La decisión de pasar a producción es humana (el PO). CI es prerequisite
Despliegue continuo	Pasa a producción automáticamente mediante pipelines, sin intervención humana

Estrategias de deployment: **Canary** (se libera primero a una pequeña parte de los usuarios y luego al resto; fallas de impacto limitado) y **Blue/Green** (dos entornos casi idénticos en producción; se transfiere el tráfico del azul (viejo) al verde (nuevo) y el azul queda para reversión). Ambas permiten actualizar sin tiempo de inactividad y revertir fácil.